

GALGOTIAS UNIVERSITY

SCHOOL OF ARTIFICIAL INTELLIGENCE

Building Future-Ready AI Talent

LABORATORY ASSIGNMENT

Course Name	Database Management System	Course Code	E2UC302B
Program	B.Tech. (AIML)	Academic Year	2026–27
Semester / Batch	3rd	Lab Assignment No.	01
Set	SET A	Date	
Faculty In-Charge	Mr. Arunendra Mani Tripathi	Max. Marks	10
Week No.	3	Duration	minutes

Course Outcome(s)	CO2: Apply SQL SELECT, WHERE and ORDER BY along with arithmetic, comparison, logical and special operators to retrieve, filter and sort data from relational databases.
Skill(s) Addressed	SQL Programming, Data Retrieval, Filtering & Sorting, Operator-based Query Formulation

1. GENERAL INSTRUCTIONS

- Attempt all questions individually.
- Use MySQL Workbench (or the prescribed SQL client) to execute queries.
- Save the SQL script as RollNo_Name_DBMS_Lab01_SETA.sql.
- Show the query and corresponding output for each task.
- Use meaningful conditions and appropriate values.
- Submit the SQL script and required screenshots through the LMS.
- Plagiarised or copied submissions will be awarded zero.

2. PROBLEM STATEMENT

Maintain employee records and retrieve/filter them using various SQL conditions and operators.

Dataset (already created in Week 1–2; re-create using the script below if not already present):

```
CREATE TABLE Employee (
    Emp_ID          INT PRIMARY KEY,
    Emp_Name        VARCHAR(30),
    Department      VARCHAR(20),
    Salary          DECIMAL(10,2),
    Joining_Date    DATE
);

INSERT INTO Employee VALUES (1, 'Rahul', 'IT', 55000, '2022-03-14');
INSERT INTO Employee VALUES (2, 'Simran', 'IT', 72000, '2021-07-01');
INSERT INTO Employee VALUES (3, 'Farhan', 'HR', 48000, '2023-01-20');
INSERT INTO Employee VALUES (4, 'Divya', 'Sales', 39000, '2020-11-05');
INSERT INTO Employee VALUES (5, 'Vikram', 'IT', NULL, '2022-09-10');
INSERT INTO Employee VALUES (6, 'Meera', 'HR', 61000, '2019-06-18');
INSERT INTO Employee VALUES (7, 'Aman', 'Sales', 45500, '2023-04-02');
INSERT INTO Employee VALUES (8, 'Anjali', 'IT', 83000, '2018-12-30');
INSERT INTO Employee VALUES (9, 'Rohit', NULL, 52000, '2021-02-14');
INSERT INTO Employee VALUES (10, 'Priya', 'Sales', 47000, '2020-08-22');
```

3. QUESTIONS / TASKS

Part 1 — *SELECT Statement, WHERE Clause, ORDER BY*

- Q1.** Display Emp_ID, Emp_Name and Salary for all employees using a column-list SELECT.
- Q2.** Display the DISTINCT Department values present in the Employee table.
- Q3.** Display all employees working in the 'IT' department using the WHERE clause.
- Q4.** Display all employees sorted in descending order of Salary, and ascending order of Emp_Name for employees with equal salary (ORDER BY, multiple columns).

Part 2 — *SQL Operators (Arithmetic, Comparison, Logical, Special)*

- Q5.** Using an arithmetic operator, display Emp_Name along with Salary increased by 8% (label the derived column Revised_Salary).
- Q6.** Using comparison and logical operators, display employees whose Salary > 50000 AND Department = 'IT', OR whose Joining_Date is after '2022-01-01'.
- Q7.** Using the BETWEEN operator, display employees whose Salary lies between 40000 and 80000.
- Q8.** Using the IN operator, display employees belonging to the department list ('IT','HR','Sales').
- Q9.** Using the LIKE and IS NULL operators, display employees whose Emp_Name starts with 'A', and separately, employees with no recorded Department.

4. STUDY MATERIAL / REFERENCE

- MySQL Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- SELECT / WHERE / ORDER BY: <https://dev.mysql.com/doc/refman/8.0/en/select.html>
- Arithmetic Operators: <https://dev.mysql.com/doc/refman/8.0/en/arithmetic-functions.html>
- Comparison Operators (incl. BETWEEN, IN, LIKE, IS NULL): <https://dev.mysql.com/doc/refman/8.0/en/comparison-operators.html>
- Logical Operators (AND, OR, NOT): <https://dev.mysql.com/doc/refman/8.0/en/logical-operators.html>
- Operator Precedence: <https://dev.mysql.com/doc/refman/8.0/en/operator-precedence.html>

5. EVALUATION RUBRIC

Evaluation Criteria	Marks
Correct column-list SELECT and DISTINCT usage	1
Correct WHERE clause (single condition)	1
Correct ORDER BY (multi-column, ASC/DESC)	1
Correct arithmetic operator producing a derived column	1
Correct combination of comparison and logical operators (AND/OR)	2
Correct use of BETWEEN and IN operators	2
Correct use of LIKE and IS NULL operators	1
Output verification, script quality and viva	1
TOTAL	10

6. EVALUATION PROCESS

- Submission check — SQL script and outputs received within the deadline.
- Execution check — queries are executed successfully and outputs are verified.
- Rubric scoring — marks are awarded against the stated criteria.
- Viva/spot-check — brief questions may be asked to verify individual understanding.

GALGOTIAS UNIVERSITY

SCHOOL OF ARTIFICIAL INTELLIGENCE

Building Future-Ready AI Talent

LABORATORY ASSIGNMENT

Course Name	Database Management System	Course Code	E2UC302B
Program	B.Tech. (AIML)	Academic Year	2026–27
Semester / Batch	3rd	Lab Assignment No.	02
Set	SET B	Date	
Faculty In-Charge	Mr. Arunendra Mani Tripathi	Max. Marks	10
Week No.	3	Duration	90 minutes

Course Outcome(s)	CO2: Apply SQL SELECT, WHERE and ORDER BY along with arithmetic, comparison, logical and special operators to retrieve, filter and sort data from relational databases.
Skill(s) Addressed	SQL Programming, Data Retrieval, Filtering & Sorting, Operator-based Query Formulation

1. GENERAL INSTRUCTIONS

- Attempt all questions individually.
- Use MySQL Workbench (or the prescribed SQL client) to execute queries.
- Save the SQL script as RollNo_Name_DBMS_Lab02_SETB.sql.
- Show the query and corresponding output for each task.
- Use meaningful conditions and appropriate values.
- Submit the SQL script and required screenshots through the LMS.
- Plagiarised or copied submissions will be awarded zero.

2. PROBLEM STATEMENT

Maintain student records and retrieve them according to academic conditions using various SQL conditions and operators.

Dataset (already created in Week 1–2; re-create using the script below if not already present):

```
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(30),
    Department VARCHAR(20),
    Semester INT,
    CGPA DECIMAL(3,2),
    Attendance DECIMAL(5,2)
);

INSERT INTO Student VALUES (1, 'Rahul', 'AIML', 4, 8.4, 88.5);
INSERT INTO Student VALUES (2, 'Anjali', 'AIML', 4, 9.1, 92.0);
INSERT INTO Student VALUES (3, 'Amit', 'CSE', 4, 7.2, 71.0);
INSERT INTO Student VALUES (4, 'Priya', 'CSE', 4, NULL, 80.0);
INSERT INTO Student VALUES (5, 'Neha', 'ECE', 4, 8.8, 90.5);
INSERT INTO Student VALUES (6, 'Rohit', 'ECE', 4, 6.9, 68.0);
INSERT INTO Student VALUES (7, 'Simran', 'AIML', 4, 8.0, 74.5);
INSERT INTO Student VALUES (8, 'Farhan', 'CSE', 4, 9.4, 95.0);
INSERT INTO Student VALUES (9, 'Divya', NULL, 4, 7.6, NULL);
INSERT INTO Student VALUES (10, 'Karan', 'AIML', 4, 8.9, 89.0);
```

3. QUESTIONS / TASKS

Part 1 — SELECT Statement, WHERE Clause, ORDER BY

- Q1.** Display Student_ID, Name and CGPA for all students using a column-list SELECT.
- Q2.** Display the DISTINCT Department values present in the Student table.
- Q3.** Display all AIML department students using the WHERE clause.
- Q4.** Display all students sorted in descending order of CGPA, and ascending order of Name for students with equal CGPA (ORDER BY, multiple columns).

Part 2 — SQL Operators (Arithmetic, Comparison, Logical, Special)

- Q5.** Using an arithmetic operator, display Name along with Attendance increased by a 2% grace margin (label the derived column Adjusted_Attendance).
- Q6.** Using comparison and logical operators, display students whose CGPA > 8.0 AND Department = 'AIML', OR whose Attendance < 75.
- Q7.** Using the BETWEEN operator, display students whose CGPA lies between 7.0 and 9.0.
- Q8.** Using the IN operator, display students belonging to the department list ('AIML','CSE','ECE').
- Q9.** Using the LIKE and IS NULL operators, display students whose Name starts with 'S', and separately, students with no recorded Attendance.

4. STUDY MATERIAL / REFERENCE

- MySQL Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- SELECT / WHERE / ORDER BY: <https://dev.mysql.com/doc/refman/8.0/en/select.html>
- Arithmetic Operators: <https://dev.mysql.com/doc/refman/8.0/en/arithmetic-functions.html>
- Comparison Operators (incl. BETWEEN, IN, LIKE, IS NULL): <https://dev.mysql.com/doc/refman/8.0/en/comparison-operators.html>
- Logical Operators (AND, OR, NOT): <https://dev.mysql.com/doc/refman/8.0/en/logical-operators.html>
- Operator Precedence: <https://dev.mysql.com/doc/refman/8.0/en/operator-precedence.html>

5. EVALUATION RUBRIC

Evaluation Criteria	Marks
Correct column-list SELECT and DISTINCT usage	1
Correct WHERE clause (single condition)	1
Correct ORDER BY (multi-column, ASC/DESC)	1
Correct arithmetic operator producing a derived column	1
Correct combination of comparison and logical operators (AND/OR)	2
Correct use of BETWEEN and IN operators	2
Correct use of LIKE and IS NULL operators	1
Output verification, script quality and viva	1
TOTAL	10

6. EVALUATION PROCESS

- Submission check — SQL script and outputs received within the deadline.
- Execution check — queries are executed successfully and outputs are verified.
- Rubric scoring — marks are awarded against the stated criteria.
- Viva/spot-check — brief questions may be asked to verify individual understanding.

GALGOTIAS UNIVERSITY

SCHOOL OF ARTIFICIAL INTELLIGENCE

Building Future-Ready AI Talent

LABORATORY ASSIGNMENT

Course Name	Database Management System	Course Code	E2UC302B
Program	B.Tech. (AIML)	Academic Year	2026–27
Semester / Batch	3rd	Lab Assignment No.	03
Set	SET C	Date	
Faculty In-Charge	Mr. Arunendra Mani Tripathi	Max. Marks	10
Week No.	3	Duration	90 minutes

Course Outcome(s)	CO2: Apply SQL SELECT, WHERE and ORDER BY along with arithmetic, comparison, logical and special operators to retrieve, filter and sort data from relational databases.
Skill(s) Addressed	SQL Programming, Data Retrieval, Filtering & Sorting, Operator-based Query Formulation

1. GENERAL INSTRUCTIONS

- Attempt all questions individually.
- Use MySQL Workbench (or the prescribed SQL client) to execute queries.
- Save the SQL script as RollNo_Name_DBMS_Lab03_SETC.sql.
- Show the query and corresponding output for each task.
- Use meaningful conditions and appropriate values.
- Submit the SQL script and required screenshots through the LMS.
- Plagiarised or copied submissions will be awarded zero.

2. PROBLEM STATEMENT

Maintain patient records and retrieve patient information using various SQL conditions and operators.

Dataset (already created in Week 1–2; re-create using the script below if not already present):

```
CREATE TABLE Patient (
  Patient_ID      INT PRIMARY KEY,
  Patient_Name    VARCHAR(30),
  Age             INT,
  Gender          CHAR(1),
  Disease         VARCHAR(30),
  Room_No        INT,
  Admission_Date  DATE
);

INSERT INTO Patient VALUES (1, 'Rehan', 54, 'M', 'Diabetes', 101, '2026-01-10');
INSERT INTO Patient VALUES (2, 'Zoya', 29, 'F', 'Asthma', 102, '2026-01-12');
INSERT INTO Patient VALUES (3, 'Ishaan', 62, 'M', 'Hypertension', 103, '2026-01-15');
INSERT INTO Patient VALUES (4, 'Tanya', 45, 'F', 'Diabetes', NULL, '2026-01-18');
INSERT INTO Patient VALUES (5, 'Aryan', 70, 'M', 'Hypertension', 104, '2026-01-20');
INSERT INTO Patient VALUES (6, 'Kavya', 38, 'F', 'Asthma', 105, '2026-01-22');
INSERT INTO Patient VALUES (7, 'Suresh', 58, 'M', 'Diabetes', 106, '2026-01-25');
INSERT INTO Patient VALUES (8, 'Meera', 33, 'F', NULL, 107, '2026-01-27');
INSERT INTO Patient VALUES (9, 'Kabir', 67, 'M', 'Hypertension', NULL, '2026-01-29');
INSERT INTO Patient VALUES (10, 'Riya', 49, 'F', 'Asthma', 108, '2026-01-30');
```

3. QUESTIONS / TASKS

Part 1 — SELECT Statement, WHERE Clause, ORDER BY

- Q1.** Display Patient_ID, Patient_Name and Disease for all patients using a column-list SELECT.
- Q2.** Display the DISTINCT Disease values present in the Patient table.
- Q3.** Display all patients suffering from 'Diabetes' using the WHERE clause.
- Q4.** Display all patients sorted in descending order of Age, and ascending order of Patient_Name for patients of equal age (ORDER BY, multiple columns).

Part 2 — SQL Operators (Arithmetic, Comparison, Logical, Special)

- Q5.** Using an arithmetic operator, display Patient_Name along with Age next year (label the derived column Age_Next_Year = Age + 1).
- Q6.** Using comparison and logical operators, display patients whose Age > 50 AND Gender = 'F', OR whose Room_No IN (101,103,106).
- Q7.** Using the BETWEEN operator, display patients whose Age lies between 30 and 60.
- Q8.** Using the IN operator, display patients whose Disease is in the list ('Diabetes','Hypertension','Asthma').
- Q9.** Using the LIKE and IS NULL operators, display patients whose Patient_Name starts with 'R', and separately, patients with no Room_No allotted.

4. STUDY MATERIAL / REFERENCE

- MySQL Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- SELECT / WHERE / ORDER BY: <https://dev.mysql.com/doc/refman/8.0/en/select.html>
- Arithmetic Operators: <https://dev.mysql.com/doc/refman/8.0/en/arithmetic-functions.html>
- Comparison Operators (incl. BETWEEN, IN, LIKE, IS NULL): <https://dev.mysql.com/doc/refman/8.0/en/comparison-operators.html>
- Logical Operators (AND, OR, NOT): <https://dev.mysql.com/doc/refman/8.0/en/logical-operators.html>
- Operator Precedence: <https://dev.mysql.com/doc/refman/8.0/en/operator-precedence.html>

5. EVALUATION RUBRIC

Evaluation Criteria	Marks
Correct column-list SELECT and DISTINCT usage	1
Correct WHERE clause (single condition)	1
Correct ORDER BY (multi-column, ASC/DESC)	1
Correct arithmetic operator producing a derived column	1
Correct combination of comparison and logical operators (AND/OR)	2
Correct use of BETWEEN and IN operators	2
Correct use of LIKE and IS NULL operators	1
Output verification, script quality and viva	1
TOTAL	10

6. EVALUATION PROCESS

- Submission check — SQL script and outputs received within the deadline.
- Execution check — queries are executed successfully and outputs are verified.
- Rubric scoring — marks are awarded against the stated criteria.
- Viva/spot-check — brief questions may be asked to verify individual understanding.

QUESTION-WISE MAPPING – SET A

Question-wise mapping with Bloom's Taxonomy Level (BTL), SOLO level, and Course Outcome (CO).

Question	BTL	SOLO Level	CO
Q1	L3 Apply	Multistructural	CO2
Q2	L3 Apply	Multistructural	CO2
Q3	L3 Apply	Relational	CO2
Q4	L3 Apply	Relational	CO2
Q5	L3 Apply	Relational	CO2
Q6	L4 Analyze	Relational	CO2
Q7	L3 Apply	Relational	CO2
Q8	L3 Apply	Relational	CO2
Q9a	L3 Apply	Relational	CO2
Q9b	L3 Apply	Relational	CO2

SOLUTION – SET A

Execute the following MySQL queries using the dataset provided in the corresponding assignment.

Q1
SELECT Emp_ID, Emp_Name, Salary FROM Employee;

Q2
SELECT DISTINCT Department FROM Employee;

Q3
SELECT * FROM Employee WHERE Department = 'IT';

Q4
SELECT * FROM Employee ORDER BY Salary DESC, Emp_Name ASC;

Q5
SELECT Emp_Name, Salary * 1.08 AS Revised_Salary FROM Employee;

Q6
SELECT * FROM Employee WHERE (Salary > 50000 AND Department = 'IT') OR Joining_Date > '2022-01-01';

Q7
SELECT * FROM Employee WHERE Salary BETWEEN 40000 AND 80000;

Q8
SELECT * FROM Employee WHERE Department IN ('IT','HR','Sales');

Q9a
SELECT * FROM Employee WHERE Emp_Name LIKE 'A%';

Q9b
SELECT * FROM Employee WHERE Department IS NULL;

Verification: Check the query output against the supplied dataset. For IS NULL queries, only records with NULL in the specified field should be returned.

QUESTION-WISE MAPPING – SET B

Question-wise mapping with Bloom's Taxonomy Level (BTL), SOLO level, and Course Outcome (CO).

Question	BTL	SOLO Level	CO
Q1	L3 Apply	Multistructural	CO2
Q2	L3 Apply	Multistructural	CO2
Q3	L3 Apply	Relational	CO2
Q4	L3 Apply	Relational	CO2
Q5	L3 Apply	Relational	CO2
Q6	L4 Analyze	Relational	CO2
Q7	L3 Apply	Relational	CO2
Q8	L3 Apply	Relational	CO2
Q9a	L3 Apply	Relational	CO2
Q9b	L3 Apply	Relational	CO2

SOLUTION – SET B

Execute the following MySQL queries using the dataset provided in the corresponding assignment.

Q1

```
SELECT Student_ID, Name, CGPA FROM Student;
```

Q2

```
SELECT DISTINCT Department FROM Student;
```

Q3

```
SELECT * FROM Student WHERE Department = 'AIML';
```

Q4

```
SELECT * FROM Student ORDER BY CGPA DESC, Name ASC;
```

Q5

```
SELECT Name, Attendance * 1.02 AS Adjusted_Attendance FROM Student;
```

Q6

```
SELECT * FROM Student WHERE (CGPA > 8.0 AND Department = 'AIML') OR Attendance < 75;
```

Q7

```
SELECT * FROM Student WHERE CGPA BETWEEN 7.0 AND 9.0;
```

Q8

```
SELECT * FROM Student WHERE Department IN ('AIML', 'CSE', 'ECE');
```

Q9a

```
SELECT * FROM Student WHERE Name LIKE 'S%';
```

Q9b

```
SELECT * FROM Student WHERE Attendance IS NULL;
```

Verification: Check the query output against the supplied dataset. For IS NULL queries, only records with NULL in the specified field should be returned.

QUESTION-WISE MAPPING – SET C

Question-wise mapping with Bloom's Taxonomy Level (BTL), SOLO level, and Course Outcome (CO).

Question	BTL	SOLO Level	CO
Q1	L3 Apply	Multistructural	CO2
Q2	L3 Apply	Multistructural	CO2
Q3	L3 Apply	Relational	CO2
Q4	L3 Apply	Relational	CO2
Q5	L3 Apply	Relational	CO2
Q6	L4 Analyze	Relational	CO2
Q7	L3 Apply	Relational	CO2
Q8	L3 Apply	Relational	CO2
Q9a	L3 Apply	Relational	CO2
Q9b	L3 Apply	Relational	CO2

SOLUTION – SET C

Execute the following MySQL queries using the dataset provided in the corresponding assignment.

Q1

```
SELECT Patient_ID, Patient_Name, Disease FROM Patient;
```

Q2

```
SELECT DISTINCT Disease FROM Patient;
```

Q3

```
SELECT * FROM Patient WHERE Disease = 'Diabetes';
```

Q4

```
SELECT * FROM Patient ORDER BY Age DESC, Patient_Name ASC;
```

Q5

```
SELECT Patient_Name, Age + 1 AS Age_Next_Year FROM Patient;
```

Q6

```
SELECT * FROM Patient WHERE (Age > 50 AND Gender = 'F') OR Room_No IN (101,103,106);
```

Q7

```
SELECT * FROM Patient WHERE Age BETWEEN 30 AND 60;
```

Q8

```
SELECT * FROM Patient WHERE Disease IN ('Diabetes','Hypertension','Asthma');
```

Q9a

```
SELECT * FROM Patient WHERE Patient_Name LIKE 'R%';
```

Q9b

```
SELECT * FROM Patient WHERE Room_No IS NULL;
```

Verification: Check the query output against the supplied dataset. For IS NULL queries, only records with NULL in the specified field should be returned.